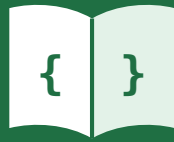


---

# Modül 03 — Web MVC ile REST API

Ders Notları

Spring Boot Edu



2026-09-23

## İçindekiler

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>1. Öğrenme Hedefleri</b>                         | <b>2</b>  |
| <b>2. Kavramlar</b>                                 | <b>2</b>  |
| 2.1 REST, HTTP Metotları ve Durum Kodları . . . . . | 2         |
| 2.2 Bir İsteğin Yolculuğu . . . . .                 | 2         |
| 2.3 DTO ve Domain Nesnesi . . . . .                 | 3         |
| <b>3. Adım Adım Örnekler</b>                        | <b>3</b>  |
| 3.1 CRUD Controller . . . . .                       | 3         |
| 3.2 Doğrulama (Validation) . . . . .                | 4         |
| 3.3 Hatalar: ProblemDetail (RFC 9457) . . . . .     | 5         |
| 3.4 Sayfalama ve Sıralama . . . . .                 | 6         |
| 3.5 API Versiyonlama (Spring Framework 7) . . . . . | 7         |
| 3.6 Content Negotiation . . . . .                   | 8         |
| 3.7 Jackson 3 ile JSON Özelleştirme . . . . .       | 9         |
| 3.8 OpenAPI ve Swagger UI . . . . .                 | 10        |
| 3.9 Virtual Thread'lerde Web İstekleri . . . . .    | 10        |
| <b>4. Sık Yapılan Hatalar ve En İyi Pratikler</b>   | <b>11</b> |
| <b>5. Özet</b>                                      | <b>12</b> |
| <b>6. İleri Okuma</b>                               | <b>12</b> |

## 1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Doğru HTTP metotları ve durum kodlarıyla bir CRUD REST API yazmak
- İstekleri Bean Validation ile doğrulamak
- Hataları RFC 9457 ProblemDetail biçiminde döndürmek
- Sayfalama ve sıralama sunmak
- Spring Framework 7'nin yerleşik **API versiyonlama** desteğini kullanmak
- Content negotiation ile aynı kaynağı farklı biçimlerde sunmak
- Jackson 3 ile JSON çıktısını özelleştirmek
- springdoc ile OpenAPI dokümanı ve Swagger UI üretmek
- Web isteklerini virtual thread'lerde çalıştırmak ve API'yi MockMvcTester ile test etmek

**Ön koşullar:** Modül 01, 02 · **Tahmini süre:** 5 saat

## 2. Kavramlar

### 2.1 REST, HTTP Metotları ve Durum Kodları

REST API'de her şey bir **kaynaktır** (/api/books/1). Ne yapılacağını HTTP metodu, sonucunu ise durum kodu söyler:

| İşlem           | Metot ve yol           | Başarılı yanıt                 |
|-----------------|------------------------|--------------------------------|
| Listele         | GET /api/books         | 200 OK                         |
| Getir           | GET /api/books/{id}    | 200 OK, yoksa 404 Not Found    |
| Oluştur         | POST /api/books        | 201 Created + Location başlığı |
| Güncelle        | PUT /api/books/{id}    | 200 OK                         |
| Sil             | DELETE /api/books/{id} | 204 No Content                 |
| Geçersiz istek  | herhangi biri          | 400 Bad Request                |
| Kural çakışması | ör. stok yok           | 409 Conflict                   |

### 2.2 Bir İsteğin Yolculuğu

1. DispatcherServlet isteği karşılar.
2. HandlerMapping, yol, metot ve **versiyona** göre doğru controller metodunu bulur.
3. Parametreler bağlanır (@PathVariable, @RequestParam, @RequestBody) ve doğrulanır.
4. Controller bir nesne döndürür. HttpMessageConverter (JSON için Jackson) onu, Accept başlığına uygun biçime çevirir.

5. Bir exception fırlarsa `@RestControllerAdvice` onu bir hata yanıtına çevirir.

## 2.3 DTO ve Domain Nesnesi

API'nin sözleşmesi (**DTO**, ör. `BookResponse`) ile iç model (`Book`) ayrı tutulur. İç modeli değiştirmek istemcileri kırmaz. İstemciyi kıran bir değişiklik gerektiğinde yeni bir API versiyonu açılır (bölüm 3.5).

## 3. Adım Adım Örnekler

Uygulamayı başlatın. Tomcat 8080 portunda açılır:

```
./mvnw -pl modules/03-web-mvc/lesson -am spring-boot:run
```

İstekleri `modules/03-web-mvc/requests.http` dosyasından IDE ile veya aşağıdaki curl komutlarıyla gönderebilirsiniz. Swagger UI: <http://localhost:8080/swagger-ui.html>

### 3.1 CRUD Controller

**Amaç:** Bir kaynağı listelemek, getirmek, oluşturmak, güncellemek ve silmek.

Controller incedir: HTTP'yi karşılar, işi `BookService`'e bırakır ve bir DTO döndürür:

```
@GetMapping(path =("/{id}", version = "1")
@Operation(summary = "Bir kitabı getir (v1) / Get a book (v1)")
public BookResponse get(@PathVariable @Positive long id) {
    return BookResponse.from(books.find(id));           // BookNotFoundException → 404
    ↪ ProblemDetail
}
```

Oluşturma işlemi 201 Created döner ve yeni kaynağın adresini Location başlığında bildirir:

```
@PostMapping
@Operation(summary = "Kitap ekle / Add a book")
public ResponseEntity<BookResponse> create(@Valid @RequestBody BookRequest request) {
    Book created = books.create(request);
    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}").buildAndExpand(created.id()).toUri();
    return ResponseEntity.created(location).body(BookResponse.from(created)); // 201 +
    ↪ Location header
}
```

**Çalıştırın:**

```
curl -s localhost:8080/api/books/1
curl -s -i -X POST localhost:8080/api/books -H 'Content-Type: application/json' \
-d '{"isbn": "978-0-13-235088-4", "title": "Clean Code", "authors": ["Robert C.
↪ Martin"], "price": 75.50, "publishedOn": "2008-08-01"}'
```

**Beklenen çıktı:**

```
{"id":1,"isbn":"9780134685991","title":"Effective Java","author":"Joshua
↪ Bloch","price":89.90,"publishedOn":"2018-01-06"}

HTTP/1.1 201
Location: http://localhost:8080/api/books/6
{"id":6,"isbn":"9780132350884","title":"Clean Code","author":"Robert C.
↪ Martin","price":75.50,"publishedOn":"2008-08-01"}
```

**Testi:** book/BookControllerTest bir web slice testidir. @WebMvcTest yalnızca MVC altyapısını ve verilen controller'ı yükler. MockMvcTester ise AssertJ ile okunur testler yazmayı sağlar:

```
@Test
void getsABook() {
    assertThat(mvc.get().uri("/api/books/1"))
        .hasStatusOk()
        .bodyJson().isLenientlyEqualTo("""
↪ {"id": 1, "isbn": "9780134685991", "title": "Effective Java",
    "author": "Joshua Bloch",
    "price": 89.90, "publishedOn": "2018-01-06"}""");
}
```

**i Not**

Spring Boot 4'te test slice'ları teknolojiye özel test starter'larındadır. @WebMvcTest için spring-boot-starter-webmvc-test gerekir.

## 3.2 Doğrulama (Validation)

**Amaç:** Geçersiz veriyi controller'a hiç sokmamak.

Kurallar DTO üzerinde durur. @ISBN, Hibernate Validator'ın ISBN kontrol basamağını da doğrulayan kısıttır:

```
public record BookRequest(
    @NotBlank @ISBN @Nullable String isbn,                // checks the
    ↪ ISBN checksum too
    @NotBlank @Size(max = 200) @Nullable String title,
    @NotEmpty @Nullable List<@NotBlank String> authors,    // constraints on
    ↪ list elements
    @NotNull @Positive @Digits(integer = 6, fraction = 2) @Nullable BigDecimal price,
    @NotNull @PastOrPresent @Nullable LocalDate publishedOn) {
}
```

Controller’da @Valid @RequestBody görevi doğrular. @PathVariable @Positive long id veya @RequestParam @Max(50) int size gibi **parametre kısıtları** ise Spring MVC’nin yerleşik metod doğrulamasıyla, ek bir ayar gerekmeden çalışır.

### Çalıştırın:

```
curl -s -X POST localhost:8080/api/books -H 'Content-Type: application/json' \
  -d '{"isbn": "123", "title": "", "authors": [], "price": -5}'
```

### Beklenen çıktı:

```
{"detail": "One or more fields are
↪ invalid", "instance": "/api/books", "status": 400, "title": "Invalid request",
"errors": [{"field": "authors", "message": "must not be
↪ empty"}, {"field": "isbn", "message": "invalid ISBN"},
{"field": "price", "message": "must be greater than 0"}, {"field": "title", "message": "must
↪ not be blank"},
{"field": "publishedOn", "message": "must not be null"}]}
```

## 3.3 Hatalar: ProblemDetail (RFC 9457)

**Amaç:** Tüm hatalar için tek, standart ve makinece okunur bir biçim kullanmak.

spring.mvc.problemDetails.enabled: true ayarıyla Spring MVC’nin kendi hataları application/problem+json olarak döner. Domain hataları için bir @RestControllerAdvice yazılır:

```
@ExceptionHandler
ProblemDetail handleBookNotFound(BookNotFoundException exception, HttpServletRequest
↪ request) {
    ProblemDetail problem = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND,
↪ exception.getMessage());
    problem.setType(URI.create("https://springbootedu.com/problems/book-not-found"));
    problem.setTitle("Book not found");
}
```

```

        problem.setInstance(URI.create(request.getRequestURI()));
        problem.setProperty("bookId", exception.bookId()); // extra, machine-readable
    ↪ field
        return problem;
    }

```

Doğrulama hatalarına hangi alanların hatalı olduğunu eklemek için, `ResponseEntityExceptionHandler`'ın ilgili metodu ezilir:

```

@Override
protected @Nullable ResponseEntity<Object> handleMethodArgumentNotValid(
    MethodArgumentNotValidException exception, HttpHeaders headers, HttpStatusCode
    ↪ status, WebRequest request) {
    ProblemDetail problem = ProblemDetail.forStatusAndDetail(status, "One or more fields
    ↪ are invalid");
    problem.setTitle("Invalid request");
    List<Map<String, String>> errors =
        ↪ exception.getBindingResult().getFieldErrors().stream()
            .map(error -> Map.of("field", error.getField(), "message",
                ↪ String.valueOf(error.getDefaultMessage())))
            .toList();
    problem.setProperty("errors", errors); // tell the client WHICH
    ↪ fields are wrong
    return handleExceptionInternal(exception, problem, headers, status, request);
}

```

**Beklenen çıktı** (`curl -s -i localhost:8080/api/books/99`):

```

HTTP/1.1 404
Content-Type: application/problem+json
{"detail":"No book with id 99","instance":"/api/books/99","status":404,"title":"Book not
    ↪ found",
  "type":"https://springbootedu.com/problems/book-not-found","bookId":99}

```

#### ★ İpucu

type, hata türünü tanımlayan kalıcı bir URI'dir. İstemciler mesaj metnine değil, type değerine göre karar vermelidir.

## 3.4 Sayfalama ve Sıralama

**Amaç:** Büyük listeleri parça parça döndürmek ve sunucuyu aşırı büyük isteklerden korumak.

```

@GetMapping
@Operation(summary = "Kitapları sayfa sayfa listele / List books page by page")
public PageResponse<BookResponse> list(
    @RequestParam(defaultValue = "0") @Min(0) int page,
    @RequestParam(defaultValue = "20") @Min(1) @Max(50) int size,           // protect
    ↪ the server
    @RequestParam(defaultValue = "title") @Pattern(regexp =
    ↪ "title|price|publishedOn") String sort,
    @RequestParam(defaultValue = "asc") @Pattern(regexp = "asc|desc") String
    ↪ direction) {
    PageResponse<Book> result = books.findPage(page, size,
    ↪ BookService.SortField.from(sort), direction.equals("desc"));
    return new PageResponse<>(result.content().stream().map(BookResponse::from).toList(),
        result.page(), result.size(), result.totalElements(), result.totalPages());
}

```

**Çalıştırın:** `curl -s 'localhost:8080/api/books?page=1&size=2&sort=price&direction=desc'`

**Beklenen çıktı:**

```

{"content":[{"id":1,...,"title":"Effective Java",...,"price":89.90,...},
            {"id":5,...,"title":"Refactoring",...,"price":85.00,...}],
"page":1,"size":2,"totalElements":5,"totalPages":3}

```

#### i Not

Burada sayfalamayı elle yazıyoruz. Modül 06'da Spring Data'nın Pageable ve Page tiplerini kullanacağız.

## 3.5 API Versiyonlama (Spring Framework 7)

**Amaç:** İstemcileri kırmadan bir API'nin yeni ve uyumsuz bir sürümünü yayınlamak.

Spring Framework 7 versiyonlamayı yerleşik olarak destekler. Boot'ta ayarlar `application.yml` içindedir:

```

mvc:
  problemdetails:
    enabled: true           # Lesson 3.3 – Spring MVC's own errors as
    ↪ application/problem+json
  apiversion:
    use:                    # Lesson 3.5 – Spring Framework 7 API versioning
      header: API-Version  # the client chooses the version with this request header
      supported: 1, 2
      default: 1           # requests without the header get version 1

```

Aynı URL'nin iki sürümü version özelliğiyle ayrılır. v1 get metodudur (bölüm 3.1), v2 ise şöyledir:

```
@GetMapping(path =("/{id}", version = "2")           // selected with the header
    ↪ "API-Version: 2"
@Operation(summary = "Bir kitabı getir (v2) / Get a book (v2)")
public BookResponseV2 getV2(@PathVariable @Positive long id) {
    return BookResponseV2.from(books.find(id));
}
```

v2'de yazarlar bir liste, fiyat ise para birimiyle birlikte bir nesnedir. Bu, v1 istemcilerini kıracak bir değişikliktir:

```
public record BookResponseV2(long id, Isbn isbn, String title, List<String> authors,
    ↪ Price price, LocalDate publishedOn) {

    public record Price(BigDecimal amount, String currency) {
    }
}
```

### Çalıştırın:

```
curl -s -H 'API-Version: 2' localhost:8080/api/books/2
curl -s -H 'API-Version: 7' localhost:8080/api/books/1
```

### Beklenen çıktı:

```
{"id":2,"isbn":"9780321336781","title":"Java Puzzlers","authors":["Joshua Bloch","Neal
    ↪ Gafter"],
  "price":{"amount":55.00,"currency":"TRY"},"publishedOn":"2005-07-04"}

{"detail":"Invalid API version:
    ↪ '7.0.0'.","instance":"/api/books/1","status":400,"title":"Bad Request"}
```

#### ★ İpucu

Versiyon başlık yerine yolda (use.path-segment), sorgu parametresinde (use.query-parameter) veya medya tipinde (use.media-type-parameter) de taşınabilir. Tek bir yöntem seçin ve tüm API'de aynı yöntemi kullanın.

## 3.6 Content Negotiation

**Amaç:** Aynı kaynağı, istemcinin Accept başlığına göre farklı biçimlerde sunmak.

produces = "text/csv" olan ikinci bir GET /api/books metodu, Accept: text/csv gönderen istemcilere seçilir:

```
@GetMapping(produces = "text/csv") // same URL, chosen by the Accept
↪ header
public String exportCsv() {
    return books.findAll().stream()
        .map(book -> book.id() + "," + book.isbn().value() + "," + book.title() + ","
            ↪ + book.price())
        .collect(Collectors.joining("\n", "id,isbn,title,price\n", "\n"));
}
```

**Çalıştırın:** `curl -s -H 'Accept: text/csv' localhost:8080/api/books`

**Beklenen çıktı:**

```
id,isbn,title,price
1,9780134685991,Effective Java,89.90
2,9780321336781,Java Puzzlers,55.00
...
```

### 3.7 Jackson 3 ile JSON Özelleştirme

**Amaç:** Bir değer nesnesinin JSON'daki görünümünü değiştirmek.

Spring Boot 4, **Jackson 3** kullanır. Paket adı `com.fasterxml.jackson` yerine `tools.jackson` olmuştur. Tarihler varsayılan olarak ISO biçiminde ("2018-01-06") yazılır. `Isbn` bir record'dur ve normalde `{"value": "..."}`  olarak yazılırdı. `@JacksonComponent` ile düz bir metin olarak yazılır ve okunur:

```
@JacksonComponent
public class IsbnJacksonComponent {

    public static class Serializer extends ValueSerializer<Isbn> {

        @Override
        public void serialize(Isbn isbn, JsonGenerator generator, SerializationContext
            ↪ context) {
            generator.writeString(isbn.value());
        }
    }

    public static class Deserializer extends ValueDeserializer<Isbn> {

        @Override
        public Isbn deserialize(JsonParser parser, DeserializationContext context) {
            return new Isbn(parser.getString()); // Jackson 3: getString() replaces
            ↪ getText()
        }
    }
}
```

```

    }
  }
}

```

**Beklenen çıktı:** Bölüm 3.1'deki yanıtta "isbn": "9780134685991".

#### ! Uyarı

Jackson 2 örneklerini kopyalarken dikkat: JsonSerializer → ValueSerializer, SerializerProvider → SerializationContext, getText() → getString() oldu ve JacksonException artık unchecked.

## 3.8 OpenAPI ve Swagger UI

**Amaç:** API'nin makinece okunur bir sözleşmesini ve etkileşimli bir dokümanını otomatik üretmek.

springdoc-openapi-starter-webmvc-ui bağımlılığı controller'ları tarar. @Tag ve @Operation açıklamaları dokümana eklenir. Genel bilgiler bir OpenAPI bean'inden gelir:

```

@Configuration(proxyBeanMethods = false)
public class OpenApiConfiguration {

    @Bean
    OpenAPI bookstoreOpenApi() {
        return new OpenAPI().info(new Info()
            .title("Kitapçı API / Bookstore API")
            .version("v1, v2 (header: API-Version)")
            .description("Spring Boot Edu – module 03"));
    }
}

```

**Açın:** <http://localhost:8080/swagger-ui.html> (arayüz) ve <http://localhost:8080/v3/api-docs> (JSON sözleşme)

**Testi:** docs/OpenApiTest

#### i Not

springdoc, Spring Boot BOM'unda değildir. Sürümü build-parent/pom.xml içinde tek bir yerde tutulur (3.1.1, Boot 4.1 için derlenmiştir).

## 3.9 Virtual Thread'lerde Web İstekleri

**Amaç:** spring.threads.virtual.enabled=true ayarının Tomcat'teki etkisini görmek.

```
@RestController
public class ThreadInfoController {

    public record ThreadInfo(String name, boolean virtual) {
    }

    @GetMapping("/api/runtime/thread")
    public ThreadInfo current() {
        Thread thread = Thread.currentThread();
        return new ThreadInfo(thread.toString(), thread.isVirtual());
    }
}
```

**Çalıştırın:** `curl -s localhost:8080/api/runtime/thread`

**Beklenen çıktı:**

```
{"name": "VirtualThread[#63,tomcat-handler-15]/runnable@ForkJoinPool-1-worker-
  ↪ 1","virtual":true}
```

**Testi:** `runtime/VirtualThreadsIT`, uygulamayı rastgele bir portta **gerçekten** başlatır ve Spring Framework 7'nin `RestTestClient`'i ile çağırır. `MockMvc` gerçek sunucu kullanmadığı için bunu test edemez.

## 4. Sık Yapılan Hatalar ve En İyi Pratikler

### !! Dikkat

Domain nesnelerini (ör. JPA entity'leri) doğrudan JSON olarak döndürmeyin. İç alanlar dışarı sızır ve iç modelde yapılan her değişiklik istemcileri kırar. Her zaman bir DTO kullanın.

- **Yapın:** Oluşturma işleminde 201 Created ve Location döndürün. Silme işleminde 204 No Content döndürün.
- **Yapmayın:** Hataları 200 OK gövdesinde `{"success": false}` olarak döndürmeyin. Doğru durum kodunu ve `ProblemDetail` kullanın.
- **Yapın:** Sayfa boyutuna bir üst sınır koyun (`@Max(50)`).
- **Yapmayın:** Controller'a iş mantığı yazmayın. Controller ince olmalı, iş servis katmanında yapılmalı.
- **Yapın:** Uyumsuz değişikliklerde yeni bir API versiyonu açın ve eski versiyonu bir süre daha destekleyin.
- **Yapmayın:** Durum tutan (in-memory) veriyi değiştiren testleri birbirine bağımlı bırakmayın. Test bağlamını `@DirtiesContext` ile yenileyin veya veriyi her testte sıfırlayın.

## 5. Özet

- REST API’de metot ve durum kodu anlam taşır. 201 + Location, 204, 404, 409 doğru kullanılmalıdır.
- Bean Validation, DTO’lar ve metot parametreleri üzerinde çalışır.
- ProblemDetail, tüm hatalar için standart bir sözleşmedir.
- Spring Framework 7, API versiyonlamayı version özelliği ve birkaç ayarla yerleşik olarak sunar.
- Content negotiation, aynı URL’den farklı biçimler sunar. Jackson 3’te özelleştirme @JacksonComponent ile yapılır.
- springdoc sözleşmeyi otomatik üretir. MockMvcTester ve RestTestClient API’yi test eder.

Sıradaki adım: [Ödevler](#)

## 6. İleri Okuma

- [Spring Framework — Request Mapping](#)
- [Spring Framework — API Versioning](#)
- [Spring Framework — Error Responses \(ProblemDetail\)](#)
- [Spring Framework — Validation](#)
- [Spring Framework — Content Negotiation](#)
- [Spring Framework — MockMvcTester](#)
- [Spring Boot — Servlet Web Applications · JSON](#)
- [RFC 9457 — Problem Details for HTTP APIs · springdoc-openapi](#)